

Stream Cipher Blind Time Long Code (SC-BLTC) Protocol Specification

1. System Parameters

The following global parameters are predefined:

- F_s (Sampling Rate): 25 kHz. This is the time-domain resolution of the digital baseband waveform.
- R_c (Chip Rate): 5 kcps. This results in a signal bandwidth of approximately 5–6 kHz (depending on the RRC roll-off factor $\alpha = 0.25$). $T_c = 1/R_c$.
- OSF (Oversampling Factor): $OSF = F_s/R_c = 5$. OSF is fixed as an integer.
- SF (Spreading Factor): 1024. The processing gain per spread symbol is $G_p = 10 \log_{10}(1024) \approx 30$ dB.
- k (Bits per Walsh Symbol): Fixed at $k = 8$.
- M_W (Walsh Orthogonal Set Size): $M_W = 2^k = 256$ (256-ary orthogonal modulation).
- R_{sym} (Spread Symbol Rate): $R_{\text{sym}} = R_c/SF = 5000/1024 \approx 4.88$ sym/s.
- R_{bit} (Post-spreading Bit Rate): $R_{\text{bit}} = k \cdot R_{\text{sym}} \approx 39.06$ bps (excluding preamble/pilot symbols and FEC overhead).
- R_{FEC} (Polar Code Rate): Used for Forward Error Correction, fixed at $1/2$. The information rate is $R_{\text{info}} = R_{\text{bit}} \times R_{\text{FEC}}$ (excluding Header/CRC/padding overhead and preamble/pilot symbol overhead).
- K_{sec} (Shared Secret Key): A 256-bit key, pre-shared between the transmitter and receiver.
- T_{epoch} (Time Epoch): Defined as the Unix Epoch (1970-01-01 00:00:00 UTC).
- IV_{res} (Time Counter Resolution): Defines the time granularity for generating the spreading code seed, fixed at 1 ms.
- **TimeIndex Search Window:** The receiver searches TI over a configured discrete window $\{TI_{\min}, TI_{\min} + 1, \dots, TI_{\min} + N_{TI} - 1\}$. Protocol operation requires the transmitter's TI_{tx} to lie inside this window.
- M (FEC Codeword Length, coded bits): Fixed value $M = 512$.
- K (FEC Information Bit Length, uncoded bits): Fixed value $K = 256$, satisfying $K = M \cdot R_{\text{FEC}}$.
- **FEC:** Uses an Arikan Polar code with $(N, K) = (512, 256)$ and CRC-aided SCL (CA-SCL) decoding.
- **CRC:** A CRC-32C is appended inside U and is used both for post-decoding verification and for CA-SCL list selection.
- **Number of Preamble Symbols:** Fixed at $N_{\text{pre}} = 2$, used for acquisition and phase ambiguity resolution.
- **Number of Data Symbols:** $N_{\text{data}} = M/k = 64$.
- **Number of Pilot Symbols:** Fixed at $N_{\text{pilot}} = 16$. Pilot symbols use W_0 and are processed by the receiver's carrier/channel tracking and verification rules (Section 4).
- **Pilot Insertion Rule:** After the preamble, the N_{data} data symbols are divided into $N_{\text{blk}} = 16$ blocks of 4 symbols each. A pilot symbol is inserted at the beginning of each block. The structure of a single block is: Pilot + Data + Data + Data + Data.
- **Number of Spread Symbols per Frame:** $N_{\text{sym}} = N_{\text{pre}} + N_{\text{pilot}} + N_{\text{data}} = 82$.
- BW_{hop} (Hopping Bandwidth): 8 kHz. The hop range is centered at baseband 0 Hz and spans ± 4 kHz.
- N_{bins} (Frequency Bins): 256. Frequency resolution is $\Delta f = BW_{\text{hop}}/N_{\text{bins}} \approx 31.25$ Hz.
- **Domain** (AES-CTR Domain Separation): Fixed 32-bit constant `0x424C5443` (ASCII "BLTC").
- L_{sym} (Spread Symbol Length): Fixed at $L_{\text{sym}} = SF = 1024$ chips for the information-bearing segment of each spread symbol.
- L_{jitter} (Guard Interval / Jitter Range): A per-symbol guard/noise segment appended after each spread symbol, with

$$j_\ell \in [40, 90] \text{ chips}, \quad \ell = 0, \dots, N_{\text{sym}} - 1.$$

This corresponds to $8 \text{ ms} \sim 18 \text{ ms}$ at $R_c = 5$ kcps.

- $T_{\text{sym,avg}}$ (Average Symbol Duration): Approximately $1024 + 65 = 1089$ chips, i.e. ≈ 217.8 ms.
- L_{frame} (Total Frame Length): Due to the per-symbol guard/noise insertion, the total frame length in chips is no longer a fixed constant. Instead,

$$L_{\text{frame}} = \sum_{\ell=0}^{N_{\text{sym}}-1} (L_{\text{sym}} + j_\ell),$$

while the symbol count remains fixed at $N_{\text{sym}} = 82$. The number of samples is $L_{\text{frame}} \cdot OSF$ (plus any optional tail padding used for filter decay).

- **E_b/N_0 Benchmark:** For a fixed $SF = 1024$, using $k = 8$ reduces the "number of chips allocated per bit", resulting in a $10 \log_{10}(k) = 9$ dB processing gain loss. The receiver employs M_W -way orthogonal Walsh

matched filtering, which provides a 3 dB equivalent advantage over binary decisions at the same E_b/N_0 . The net change is a 6 dB degradation.

- **Walsh Code Definition:** Uses the row vectors $W_m[j] \in \{+1, -1\}$ of the order- $SF = 1024$ Sylvester-Hadamard matrix, where

$$W_m[j] = (-1)^{\text{popcount}(m \& j) \bmod 2}, \quad m \in [0, 1023], j \in [0, 1023]$$

This protocol only uses the first M_W orthogonal rows for $m \in [0, 255]$. $W_0[j] \equiv +1$.

- **RRC Pulse Shaping:**

- Roll-off: $\alpha = 0.25$.
- Span: $S_{\text{rrc}} = 6$ symbols.
- Taps: $L_{\text{rrc}} = S_{\text{rrc}} \cdot OSF + 1 = 31$.
- Group delay (samples): $D_{\text{rrc}} = (L_{\text{rrc}} - 1)/2 = 15$.

- **Tail and Ramp-Down:**

- Tail padding: $N_{\text{tail}} = 8$ symbols (each symbol is SF chips).
- Ramp-down duration: $T_{\text{ramp}} = 20$ ms.

- **Guard/Noise Filling:**

- Guard/noise chip power relative to data chips: -3 dB.
- Guard soft-ramp length: $L_{\text{GI,ramp}} = 8$ chips (capped by $j_\ell/2$ per symbol).

- **Acquisition Parameters:**

- Acquisition FFT size: $N_{\text{FFT}} = 32768$.
- CFO search half-bandwidth: $f_{\text{search}} = 8000$ Hz.
- Hybrid verification threshold multiplier: $\gamma_{\text{hyb}} = 10$.
- RAKE search half-window: $T_{\text{rake}} = 4$ ms.
- Pilot timing verification window: ± 32 samples.

2. Core Primitive: Cryptographically Secure Spreading Sequence Generator (CSPRNG Spreading)

AES-CTR mode is used as the Pseudo-Random Number Generator (PRNG).

Generation Function $\text{GenCode}(\text{Key}, \text{TimeIndex}, N_{\text{sym}})$

Inputs:

- *Key*: The shared secret key.
- *TimeIndex*: An integer time counter for the transmission instant.
- N_{sym} : The number of spread symbols per frame (fixed at 82).

Outputs:

1. Cryptographic chip masks C_{seq} for de-spreading, organized as N_{sym} blocks of length $SF = 1024$ chips.
2. Frequency control sequence $F_{\text{seq}} = \{f_0, f_1, \dots, f_{N_{\text{sym}}-1}\}$ (Hz).
3. Jitter/guard control sequence $J_{\text{seq}} = \{j_0, j_1, \dots, j_{N_{\text{sym}}-1}\}$ (chips).

Process:

1. **Construct Nonce/Counter:** Follows the standard CTR “Nonce + Counter” structure.
 - The 96-bit Nonce is defined as $\text{Nonce} = (\text{TimeIndex} \parallel \text{Domain})$, where *TimeIndex* is 64-bit big-endian and *Domain* is a 32-bit big-endian constant `0x424C5443` (ASCII for “BLTC”) used for domain separation.

- A 32-bit BlockCounter starts from 0 and increments for each block, forming the AES block input $IV_{\text{block}} = (\text{Nonce} \parallel \text{BlockCounter})$.
2. **AES-CTR Generation:** Encrypt an all-zero data stream to produce the keystream bytes.
 3. **Structured Consumption (Per Symbol):** For the ℓ -th spread symbol ($\ell = 0, \dots, N_{\text{sym}} - 1$), consume the keystream as:

1. **Data Block:** Take the next 1024 bits and map them (MSB-first per byte) to the bipolar chip mask $C_{\text{seq}}[\ell][j] \in \{+1, -1\}$ using $0 \rightarrow +1, 1 \rightarrow -1$.
2. **Control Block:** Take the next 16 bits as metadata:
 - High 8 bits: $V_{\text{freq}} \in [0, 255]$
 - Low 8 bits: $V_{\text{jitter}} \in [0, 255]$
3. **Parameter Mapping:**
 - Hop offset:

$$f_{\ell} = \left(\frac{V_{\text{freq}}}{256.0} - 0.5 \right) \cdot BW_{\text{hop}} \quad (\text{Hz}).$$

- Guard length:

$$j_{\ell} = \begin{cases} 40 + \left\lfloor \frac{V_{\text{jitter}} \cdot 50}{256} \right\rfloor & V_{\text{jitter}} \in [0, 254] \\ 40 + 50 & V_{\text{jitter}} = 255 \end{cases} \quad (\text{chips}).$$

This yields $j_{\ell} \in [40, 90]$.

4. **Endianness and Bit Order Convention:**

- **TimeIndex** is encoded as **64-bit big-endian**.
 - **Domain** and **BlockCounter** are encoded as **32-bit big-endian**.
 - The AES-CTR output bytes are expanded into a bitstream with **bitorder=big** (MSB-first for each byte).
5. **Non-Reuse Constraint:** Under a fixed shared key K_{sec} , each transmission uses a unique *TimeIndex* and the transmitter does not transmit twice with the same *TimeIndex*. With $\text{TimeIndex} = \lfloor t_{\text{tx}} / IV_{\text{res}} \rfloor$, consecutive transmissions are separated by at least IV_{res} in t_{tx} to ensure distinct *TimeIndex* values.

3. Transmitter Protocol

The transmitter derives TI_{tx} from its local time source and emits a single frame. The receiver searches TI over its configured window (Section 1).

Step A0: Frame Format

This protocol uses a fixed **symbol count** per frame ($N_{\text{sym}} = 82$). Due to per-symbol guard/noise insertion, the **total sample length** of a frame is variable and is determined by the generated J_{seq} for the frame's *TimeIndex*.

- **Information Bits (uncoded):** U , with a length of $K = 256$ bits, composed of the following parts:

1. **Header (plaintext/structural fields):**
 - **Ver:** Protocol Version (4 bits).
 - **Type:** Message Type (4 bits).
 - **Len:** Payload Length (8 bits, in Bytes).
2. **Payload:** Service data bits.
3. **CRC:** A CRC-32C is computed over the Header+Payload for post-decoding verification.
4. **Padding:** If Header+Payload+CRC does not fill up K bits, it is padded with zeros.
5. **Byte/Bit Order:**
 - U is constructed as a 32-byte array: `Header(2) || Payload(Len) || CRC32C(4) || Pad(0)` to 32 bytes.
 - Bytes are expanded MSB-first into bits.
 - CRC32C is appended as 4 bytes in big-endian byte order.

- **Polar Encoding:** U is encoded with a fixed code rate $R_{\text{FEC}} = 1/2$ into a codeword B of length $M = 512$ (coded bits) using a fixed Polar code $(N, K) = (512, 256)$.

Frozen Set and Bit Mapping:

1. Construct a length- N vector $\mathbf{u} = \{u_0, \dots, u_{N-1}\}$ with a fixed frozen set $\mathcal{F}$ of size $N - K$. For all $i \in \mathcal{F}$, set $u_i = 0$. For all $i \notin \mathcal{F}$ (the information set), fill u_i with the next bit from U in increasing index order.
2. The frozen set $\mathcal{F}$ is determined using Polarization Weight (PW) ranking for $N = 512$ with $\beta = 2^{1/4}$:

$$w(i) = \sum_{j=0}^{\log_2 N - 1} b_j(i) \beta^j, \quad b_j(i) \in \{0, 1\} \text{ is bit } j \text{ of } i.$$

The K indices with the largest $w(i)$ form the information set; all other indices are frozen.

3. Apply the standard Arikan Polar transform to obtain the codeword $\mathbf{B} = \{b_0, \dots, b_{N-1}\}$.

- **Full-Frame Interleaving:** Apply a fixed bit permutation $\pi(\cdot)$ over the full codeword to produce an interleaved sequence $B^{(\pi)} = \{b_0^{(\pi)}, \dots, b_{M-1}^{(\pi)}\}$:

$$b_j^{(\pi)} = b_{\pi(j)}, \quad \pi(j) = (A \cdot j + B) \bmod M, \quad A = 109, B = 37.$$

For $M = 2^9$, any odd A yields a bijection; $A = 109$ is odd.

- **Symbol Mapping (256-ary Walsh):** $B^{(\pi)}$ is grouped into $N_{\text{data}} = 64$ symbol indices $m_q \in [0, 255]$ of $k = 8$ bits each (MSB first):

$$m_q = \sum_{t=0}^{k-1} b_{qk+t}^{(\pi)} \cdot 2^{k-1-t}, \quad q = 0, \dots, N_{\text{data}} - 1$$

These N_{data} indices are mapped to N_{data} orthogonal Walsh spread symbols.

- **Symbol Arrangement (Preamble + Pilots + Data):**

- Spread symbols are numbered $\ell = 0, \dots, N_{\text{sym}} - 1$.
- $\ell = 0$ and $\ell = 1$ are the two Preamble symbols.
- Following the preamble are $N_{\text{blk}} = 16$ blocks, numbered $r = 0, \dots, 15$. Each block consists of 5 spread symbols: 1 pilot + 4 data.
- The position of the pilot symbol for block r is

$$\ell_{\text{pilot}}(r) = 2 + 5r$$

- The position of the q -th data symbol ($q = 0, \dots, 63$) is determined as follows: let $r = \lfloor q/4 \rfloor$ and $s = q \bmod 4$, then

$$\ell_{\text{data}}(q) = 2 + 5r + 1 + s$$

Step A: Payload Preparation

Assemble the frame according to Step A0 to get the fixed-length information bits U (including CRC and padding). Perform Polar encoding on U to get the codeword bit sequence $B = \{b_0, \dots, b_{M-1}\}$. Apply the full-frame interleaver to get $B^{(\pi)}$, and pack it into a sequence of symbol indices $\{m_0, \dots, m_{N_{\text{data}}-1}\}$ using $k = 8$ bits per symbol.

Step B: Determine Transmission Time Seed

Read the current local time t_{tx} . Calculate the time counter:

$$TI_{\text{tx}} = \left\lfloor \frac{t_{\text{tx}}}{IV_{\text{res}}} \right\rfloor$$

The receiver estimates TI_{tx} via blind acquisition (Section 4).

Step C: Generate Full-Frame Spreading Code

Call the generation function:

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \text{GenCode}(K_{\text{sec}}, TI_{\text{tx}}, N_{\text{sym}})$$

where: - C_{seq} provides the N_{sym} per-symbol chip masks (each of length $SF = 1024$). - F_{seq} provides the per-symbol hop offsets f_ℓ (Hz). - J_{seq} provides the per-symbol guard lengths j_ℓ (chips).

Step D: Walsh Orthogonal Spreading Modulation

The cryptographic chip sequence is used as a “mask/scrambling code” and is combined with the orthogonal Walsh rows to generate the transmitted chips for each spread symbol.

The chips for the ℓ -th spread symbol ($\ell = 0, \dots, N_{\text{sym}} - 1$) are denoted as $S[\ell \cdot SF + j]$, where $0 \leq j < SF$.

- **Preamble Symbols** ($\ell = 0, 1$): Fixed to use a Barker-2 synchronization word with Walsh index 0, i.e., $[+W_0, -W_0]$:

– **Preamble symbol 0** ($\ell = 0, +W_0$):

$$S[0 \cdot SF + j] = C_{\text{seq}}[0 \cdot SF + j]$$

– **Preamble symbol 1** ($\ell = 1, -W_0$):

$$S[1 \cdot SF + j] = -C_{\text{seq}}[1 \cdot SF + j]$$

These two preamble symbols are used for acquisition and phase ambiguity resolution and do not carry information bits.

- **Pilot Symbols** ($\ell = \ell_{\text{pilot}}(r)$, $r = 0, \dots, 15$): Fixed to use $W_0[j] \equiv +1$:

$$S[\ell \cdot SF + j] = C_{\text{seq}}[\ell \cdot SF + j]$$

Pilot symbols are used for continuous phase tracking and cycle slip suppression and do not carry information bits.

- **Data Symbols** ($\ell = \ell_{\text{data}}(q)$, $q = 0, \dots, N_{\text{data}} - 1$): Uses the m_q -th Walsh row. Let $\ell = \ell_{\text{data}}(q)$:

$$S[\ell \cdot SF + j] = W_{m_q}[j] \cdot C_{\text{seq}}[\ell \cdot SF + j]$$

This defines the information-bearing chip segment of each spread symbol (length $SF = 1024$ chips). The transmitted chip stream is formed by appending a per-symbol guard/noise segment after each spread symbol (Step D1.5), resulting in a variable-length chip stream.

Step D1.5: Guard Interval Insertion and Noise Filling

After the ℓ -th spread symbol's 1024 data chips, append a length- j_ℓ guard/noise segment:

1. **Data Segment:** Send the $SF = 1024$ chips defined in Step D.
2. **Guard/Noise Segment:** Immediately follow with j_ℓ real-valued chips of zero-mean Gaussian noise.
 - Noise power relative to the unit-power data chips is fixed to -3 dB, i.e. $\sigma^2 = 10^{-3/10}$ and $\sigma = \sqrt{10^{-3/10}}$.
 - This segment is not Walsh-spread and carries no information.
 - The guard/noise chips are generated deterministically per frame and per symbol:

1. Seed:

$$s_{\ell,0} = (TI_{tx} \oplus (\ell \ll 32) \oplus 0xA7F031D359B29C11) \oplus 0x9E3779B97F4A7C15.$$

2. PRNG core (xorshift64*): with state s ,

$$x \leftarrow s; \ x \leftarrow x \oplus (x \gg 12); \ x \leftarrow x \oplus (x \ll 25); \ x \leftarrow x \oplus (x \gg 27); \ s \leftarrow x; \ u_{64} \leftarrow x \cdot 0x2545F4914F6CDD1D.$$

3. Convert to uniforms in $(0, 1)$:

$$u = \frac{(u_{64} \gg 40) + 1}{2^{24} + 2}.$$

4. Box-Muller: draw (u_1, u_2) and compute

$$z_0 = \sqrt{-2 \ln(\max(u_1, 10^{-12}))} \cos(2\pi u_2), \quad z_1 = \sqrt{-2 \ln(\max(u_1, 10^{-12}))} \sin(2\pi u_2).$$

The generator outputs z_0, z_1 alternately and scales by σ .

- Guard soft ramp: let $r_\ell = \min(L_{GI, \text{ramp}}, \lfloor j_\ell/2 \rfloor)$. If $r_\ell \geq 2$, multiply the first r_ℓ and last r_ℓ guard chips by the half-cosine window

$$w_{GI}[k] = \frac{1}{2} \left(1 - \cos \left(\pi \frac{k}{r_\ell - 1} \right) \right), \quad k = 0, \dots, r_\ell - 1.$$

Step D2: Pulse Shaping and Oversampling

The baseband chip sequence is pulse-shaped.

- **Shaping Filter:** A Root-Raised-Cosine (RRC) filter $g_{\text{rrc}}(t)$ is used.
- **Oversampling:** The shaped waveform is output at a sampling rate of $F_s = OSF \cdot R_c$.

Let $S_{\text{base}}[n]$ be the RRC-shaped complex baseband waveform at the sampling rate, generated from the chip stream that includes both the data segments and the guard/noise segments. Its length is $L_{\text{frame}} \cdot OSF$ (plus any optional tail padding used for filter decay).

RRC pulse shaping is applied before digital frequency rotation (Step D3). The filter taps are generated by the standard continuous-time RRC closed form sampled at $1/F_s$ and normalized to unit energy:

- Let $a = \alpha$ and define t in chip periods. For each tap index $i = 0, \dots, L_{\text{rrc}} - 1$, define

$$t_i = \frac{i - (L_{\text{rrc}} - 1)/2}{OSF}.$$

- The unnormalized impulse response is

$$h(t) = \frac{\sin(\pi t(1 - a)) + 4at \cos(\pi t(1 + a))}{\pi t(1 - (4at)^2)}.$$

Special cases:

– At $t = 0$:

$$h(0) = 1 - a + \frac{4a}{\pi}.$$

– At $t = \pm \frac{1}{4a}$:

$$h\left(\pm \frac{1}{4a}\right) = \frac{a}{\sqrt{2}} \left[\left(1 + \frac{2}{\pi}\right) \sin\left(\frac{\pi}{4a}\right) + \left(1 - \frac{2}{\pi}\right) \cos\left(\frac{\pi}{4a}\right) \right].$$

- Sample the taps as $h[i] = h(t_i)$ and normalize to unit energy:

$$h[i] \leftarrow \frac{h[i]}{\sqrt{\sum_{k=0}^{L_{\text{rrc}}-1} h[k]^2}}.$$

Chip upsampling is performed by placing each real chip at sample index $n = c \cdot OSF$ and inserting zeros at the intermediate samples. The pulse-shaped sequence is produced by causal FIR convolution

$$y[n] = \sum_{k=0}^{L_{\text{rrc}}-1} h[k] \cdot x[n - k].$$

Step D3: Digital Frequency Rotation and Phase Continuity

Hopping is performed by complex rotation at baseband with a cumulative phase.

Let f_ℓ be the hop offset for symbol ℓ (from F_{seq}). Define a cumulative phase $\Phi[n]$ and a per-sample phase increment:

$$\Delta\omega_\ell = 2\pi \frac{f_\ell}{F_s}.$$

For samples n that fall within the ℓ -th symbol's **time span** (including its guard/noise segment), update:

$$\Phi[n] = \Phi[n-1] + \Delta\omega_\ell.$$

Transmit waveform:

$$S_{\text{tx}}[n] = S_{\text{base}}[n] \cdot e^{j\Phi[n]}.$$

Important (Phase Continuity): $\Phi[n]$ must remain cumulative across symbol boundaries (do not reset at symbol transitions). This ensures smooth time-domain transitions at hop changes and strongly suppresses spectral splatter.

Step E: Transmission

This specification defines the complex baseband waveform $S_{\text{tx}}[n]$ at sampling rate F_s . Any RF upconversion and emission are outside the scope of this document.

Step E2: Soft Shutdown

1. **Tail Padding:** After transmitting the N_{sym} symbols, append an additional $N_{\text{tail}} = 8$ all-zero symbols (Zero Padding).
2. **Power Ramp-Down:** Let $L_{\text{ramp}} = \text{round}(F_s \cdot T_{\text{ramp}})$ samples. Multiply the last L_{ramp} samples by the half-cosine ramp

$$w_{\text{tx}}[i] = \frac{1}{2} \left(1 + \cos \left(\pi \frac{i}{L_{\text{ramp}} - 1} \right) \right), \quad i = 0, \dots, L_{\text{ramp}} - 1,$$

applied as $S_{\text{tx}}[N - L_{\text{ramp}} + i] \leftarrow S_{\text{tx}}[N - L_{\text{ramp}} + i] \cdot w_{\text{tx}}[i]$.

4. Receiver Protocol

The receiver does not know TI_{tx} and searches over a configured window $\{TI_{\text{min}}, \dots, TI_{\text{min}} + N_{TI} - 1\}$. If this window is derived from a local-time estimate t_{rx} and an uncertainty bound W , compute:

$$TI_{\text{min}} = \left\lfloor \frac{t_{\text{rx}} - W}{IV_{\text{res}}} \right\rfloor, \quad N_{TI} = \left\lfloor \frac{t_{\text{rx}} + W}{IV_{\text{res}}} \right\rfloor - TI_{\text{min}} + 1.$$

Step A: Signal Buffering

The receiver continuously records the signal into a circular buffer **Buffer**.

Step B: Blind Acquisition (De-hopping + FFT)

Acquisition is performed under a TI hypothesis and reconstructs the corresponding hop/guard structure from **GenCode**.

The receiver first down-converts the signal to complex baseband I/Q. The acquisition stage operates directly on these **raw baseband samples**.

4.B.1 Search Space Definition

- **Time Hypothesis:** Iterate $TI_{\text{search}} \in \{TI_{\text{min}}, \dots, TI_{\text{min}} + N_{TI} - 1\}$.
- **Intra-Epoch Starting Offset:** Define $N_{\text{IV,samp}} = \text{round}(F_s \cdot IV_{\text{res}})$. Coarse acquisition evaluates offsets on a fixed stride of 4 samples, i.e. $n_{\text{offset,total}} \in \{0, 4, 8, \dots\} < N_{\text{IV,samp}}$. The final RAKE timing refinement (4.B.2 Step 6) evaluates a contiguous sample-level window.
- **Residual Frequency Offset Search:** Use an FFT peak search within $|\Delta f| \leq f_{\text{search}}$, with $f_{\text{search}} = 8000$ Hz.

4.B.2 Algorithm Outline For each TI_{search} :

1. **Generate Schedules:**

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \text{GenCode}(K_{\text{sec}}, TI_{\text{search}}, N_{\text{sym}}).$$

2. **Compute Symbol Start Positions:** Using J_{seq} , compute the symbol start positions (in samples) relative to the frame start:

$$n_{\ell} = \left(\sum_{k=0}^{\ell-1} (SF + j_k) \right) \cdot OSF.$$

3. **De-hopping Rotation (Time-Frequency Grid Reconstruction):** Construct a de-hopping phasor whose instantaneous frequency follows the hop sequence and remains phase-continuous:

$$D[n] = e^{-j\Phi[n]}, \quad \Phi[n] = \Phi[n-1] + 2\pi \frac{f_{\ell}}{F_s},$$

for samples n within symbol ℓ 's time span (data + guard/noise).

4. **Preamble FFT Detection:** For each start-offset hypothesis $n_{\text{offset,total}}$, extract a window covering **both** preamble symbols' **data segments** and coherently accumulate them for the coarse FFT score. Concretely, form a sparse window

$$y[n] = \begin{cases} x[n] \cdot s_0^*[n] & n \in [0, SF \cdot OSF) \\ 0 & n \in [SF \cdot OSF, (SF + j_0) \cdot OSF) \\ x[n] \cdot s_1^*[n - (SF + j_0) \cdot OSF] & n \in [(SF + j_0) \cdot OSF, (2SF + j_0) \cdot OSF) \end{cases}$$

where s_0, s_1 are the locally generated (hopped) TX-shaped preamble references for symbol 0 and 1 (with the Barker-2 sign pattern $+W_0, -W_0$) and the hop phasor is phase-continuous across the symbol boundary. Concretely, if symbol 0 uses hop f_0 and the time between the two preamble symbol starts is $\Delta n_1 = (SF + j_0) \cdot OSF$ samples, then the symbol-1 reference includes the accumulated phase factor $e^{+j2\pi f_0 \Delta n_1 / F_s}$ at its start.

The receiver pads $y[n]$ to $N_{\text{FFT}} = 32768$ and computes an FFT. Let $Y[k]$ be the FFT output. Define the bin limit

$$k_{\text{max}} = \left\lfloor \frac{\min(f_{\text{search}}, F_s/2) N_{\text{FFT}}}{F_s} \right\rfloor, \quad k_{\text{max}} \in [1, N_{\text{FFT}}/2 - 1].$$

The coarse CFO bin is selected by maximizing power over bins $k \in \{0, 1, \dots, k_{\text{max}}\} \cup \{N_{\text{FFT}} - k_{\text{max}}, \dots, N_{\text{FFT}} - 1\}$:

$$k_0 = \arg \max_k |Y[k]|^2.$$

The selected bin is mapped to a signed bin index $b_0 \in [-k_{\text{max}}, k_{\text{max}}]$ (with negative frequencies represented by the upper FFT bins). A quadratic interpolation refines the bin location when $|b_0| < k_{\text{max}}$:

$$\delta = \frac{1}{2} \frac{P_{-1} - P_{+1}}{P_{-1} - 2P_0 + P_{+1}}, \quad b = b_0 + \delta,$$

where $P_{-1} = |Y[b_0 - 1]|^2$, $P_0 = |Y[b_0]|^2$, $P_{+1} = |Y[b_0 + 1]|^2$ using circular indexing on the FFT output. The coarse CFO estimate is

$$\widehat{\Delta f}_{\text{coarse}} = b \cdot \frac{F_s}{N_{\text{FFT}}}.$$

5. **Verification Using Pilots:** For top candidates, refine CFO on a small grid and verify using noncoherent pilot energy. Pilot symbol positions are computed using n_ℓ , and the receiver integrates only over each pilot's 1024-chip data segment (ignoring the guard/noise segment).

- Noise power estimate: let σ^2 be estimated from the raw baseband as

$$\widehat{\sigma^2} = \frac{\text{median}(|x[n]|^2)}{\ln 2}.$$

- Fine CFO search: for each candidate coarse CFO $\widehat{\Delta f}_{\text{coarse}}$, evaluate

$$\widehat{\Delta f} = \arg \max_{\Delta f \in \{\widehat{\Delta f}_{\text{coarse}} + q \cdot 0.25 \text{ Hz} : q = -8, \dots, 8\}} \left| \sum_n y[n] \cdot e^{-j2\pi \Delta f n / F_s} \right|^2.$$

- Pilot verification: for each pilot symbol $r = 0, \dots, N_{\text{pilot}} - 1$ at index $\ell_{\text{pilot}}(r) = 2 + 5r$, build the TX-shaped hopped pilot reference $p_r[n]$ for the pilot's 1024-chip data segment and evaluate a small timing window $\Delta n \in [-32, +32]$ samples around the predicted start $n_{\ell_{\text{pilot}}(r)}$:

$$E_r = \max_{\Delta n \in [-32, 32]} \left| \sum_{n=0}^{SF \cdot OSF - 1} x[n_0 + n_{\ell_{\text{pilot}}(r)} + \Delta n + n] \cdot p_r^*[n] \cdot e^{-j2\pi \widehat{\Delta f} n / F_s} \right|^2.$$

The pilot verification energy is $V_{\text{pilot}} = \sum_r E_r$.

- Preamble verification energy: V_{pre} is the best fine-CFO preamble energy from the previous step.
- Acceptance threshold: define reference energies E_{pre} (two preamble references) and E_{sym} (one pilot reference) and compute

$$E_{\text{tot}} = E_{\text{pre}} + N_{\text{pilot}} E_{\text{sym}}, \quad \Gamma = \gamma_{\text{hyb}} \cdot \widehat{\sigma^2} \cdot E_{\text{tot}}, \quad \gamma_{\text{hyb}} = 10.$$

The candidate is accepted if $V_{\text{pre}} + V_{\text{pilot}} > \Gamma$. Among accepted candidates, the receiver selects the one with the largest $V_{\text{pre}} + V_{\text{pilot}}$ (tie-break: smaller $(TI, n_{\text{offset, total}})$).

6. **Timing Refinement and RAKE Initialization:** Refine the start offset by coherent correlation around the coarse estimate. For better timing discrimination under multipath, the receiver jointly uses **both** preamble symbols, taking into account the guard length j_0 between them.

Concretely, for each candidate start offset $\hat{n}$ in the search window (around the coarse estimate), form the two complex correlations

$$z_0(\hat{n}) = \sum_{n=0}^{SF \cdot OSF - 1} x[\hat{n} + n] \cdot s_0^*[n],$$

$$z_1(\hat{n}) = \sum_{n=0}^{SF \cdot OSF - 1} x[\hat{n} + (SF + j_0) \cdot OSF + n] \cdot s_1^*[n],$$

where s_0, s_1 are the locally generated TX-shaped preamble references (with the Barker-2 sign pattern and phase-continuous hopping, as in Step 4).

The timing score is the **noncoherent** sum of the two per-symbol correlation energies:

$$e_0(\hat{n}) = |z_0(\hat{n})|^2, \quad e_1(\hat{n}) = |z_1(\hat{n})|^2, \quad \Lambda(\hat{n}) = e_0(\hat{n}) + e_1(\hat{n}).$$

Select the refined main-path start:

$$\hat{n}_0 = \arg \max_{\hat{n}} \Lambda(\hat{n}) \quad (\text{tie-break: smaller } \hat{n}).$$

The receiver constructs the returned RAKE finger list $\{\hat{n}_{\text{finger}, i}\}$ directly from these hypotheses:

1. Sort all hypotheses by descending $\Lambda(\hat{n})$ (tie-break: smaller $\hat{n}$).
2. Initialize the finger list with $\hat{n}_{\text{finger}, 0} = \hat{n}_0$.
3. Iterate the remaining hypotheses in sorted order and append $\hat{n}$ to the finger list iff:
 - $\hat{n} \geq \hat{n}_0$ (non-negative delay relative to the main path), and

- $|\hat{n} - \hat{n}_{\text{finger},k}| \geq OSF$ for all already-selected fingers k (minimum separation).
4. Stop when either:
- the list length reaches $N_{\text{finger},\text{out}} = \text{clamp}(N_{\text{finger},\text{req}}, 1, 16)$, or
 - the hypothesis list is exhausted.

The acquisition output parameters are: $\widehat{TI}_{\text{tx}}$, the main path starting position $\hat{n}_0$, the refined CFO estimate $\widehat{\Delta f}$, an **ordered list** of RAKE finger starting positions $\{\hat{n}_{\text{finger},i}\}_{i=0}^{N_{\text{finger},\text{out}}-1}$ (with $\hat{n}_{\text{finger},0} = \hat{n}_0$), and the verification statistic p_{max} .

Step C: RAKE Reception & Carrier Synchronization

Using the acquisition outputs $(\widehat{TI}_{\text{tx}}, \hat{n}_0, \widehat{\Delta f}, \{\hat{n}_{\text{finger},i}\})$, perform the following preprocessing on the **raw** base-band samples:

1. **Post-Acquisition Sample-Offset Refinement:** Refine the sample start position by searching a small integer window $\Delta n \in [-6, 6]$ around $\hat{n}_0$ (and apply the same Δn to all $\{\hat{n}_{\text{finger},i}\}$). A sample-index error Δn introduces the hop-dependent phase error

$$\Delta\phi_\ell \approx 2\pi \frac{f_\ell}{F_s} \Delta n,$$

evaluated per spread symbol ℓ . The receiver evaluates the refinement candidates in the fixed order:

$$\Delta n \in \{0, -1, +1, -2, +2, \dots, -6, +6\}.$$

For each candidate Δn , the receiver applies the same shift to every finger offset $\{\hat{n}_{\text{finger},i}\}$, then performs CFO derotation, de-hopping, matched filtering, and a full-frame demod/decode.

- If any Δn yields a CRC-valid decode, the receiver selects the **first** such Δn in the order above and returns that decode result.
- If no Δn yields a CRC-valid decode, the receiver selects the Δn that maximizes the verification statistic.

$$V(\Delta n) = V_{\text{pre}}(\Delta n) + V_{\text{pilot}}(\Delta n),$$

where

$$V_{\text{pre}}(\Delta n) = \sum_i \left| \left(\sum_{j=0}^{SF-1} \mathbf{u}_{i,0}[j] \right) - \left(\sum_{j=0}^{SF-1} \mathbf{u}_{i,1}[j] \right) \right|^2,$$

$$V_{\text{pilot}}(\Delta n) = \sum_{\ell \in \mathcal{P}} |z_\ell|^2,$$

$\mathcal{P}$ is the set of pilot symbol indices, and z_ℓ is the PLL prompt sum used in Step C.2(a) for preamble/pilots.

2. Apply a sample-rate CFO derotation using $\widehat{\Delta f}$:

$$x'[n] = x[n] \cdot e^{-j2\pi \widehat{\Delta f} n / F_s}.$$

3. Reconstruct $(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}})$ from $\widehat{TI}_{\text{tx}}$ (via **GenCode**) and apply a phase-continuous sample-rate **de-hopping** rotation to $x'[n]$ using F_{seq} and J_{seq} (Step 4.B.2). The de-hopping rotation starts at the absolute frame start sample and also rotates the prefix samples that feed the matched filter using the first hop frequency.
4. Apply the receive RRC matched filter (same taps as the transmitter) using the causal convolution $y[n] = \sum_k h[k] \cdot x'[n-k]$ to obtain the matched-filtered sequence $y[n]$.
 - The RRC group delay is D_{rrc} samples (Section 1).

- The cascade (TX+RX) group delay is $D_{\text{cas}} = 2D_{\text{rrc}}$ samples. Symbol sampling times are referenced to the frame start plus D_{cas} .

All subsequent RAKE/DLL/PLL processing in Step C/D operates on this matched-filtered, de-hopped signal.

0. Full-Frame Local Code Generation

Use the acquired $\widehat{TI}_{\text{tx}}$ to call

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \text{GenCode}(K_{\text{sec}}, \widehat{TI}_{\text{tx}}, N_{\text{sym}})$$

to generate the full-frame cryptographic chip masks (for de-masking) as well as the hop/guard schedules (for de-hopping and symbol windowing).

1. RAKE Structure

The acquisition stage (4.B.2–4.B.3) outputs an ordered list of RAKE finger starting positions $\{\hat{n}_{\text{finger},i}\}_{i=0}^{N_{\text{finger,out}}-1}$ (in samples), with $\hat{n}_{\text{finger},0} = \hat{n}_0$. The list is ordered by descending preamble correlation energy (tie-break: smaller offset).

Use at most three fingers for tracking and MRC combining:

$$N_{\text{finger}} = \min(3, N_{\text{finger,out}}).$$

Define the per-finger integer delay (samples) relative to the main path:

$$\Delta d_i = \hat{n}_{\text{finger},i} - \hat{n}_{\text{finger},0}.$$

All RAKE/DLL/PLL/MRC processing in Step C uses only the first N_{finger} offsets and the corresponding Δd_i .

Window Gating (Data vs Guard/Noise)

For each spread symbol ℓ , the receiver knows: - the **data** segment length is fixed at 1024 chips, - the following **guard/noise** segment length is j_ℓ chips (from J_{seq}).

The RAKE receiver integrates only over the 1024-chip data window. During the j_ℓ guard/noise interval, the receiver does not update its correlators.

2. Dual-Loop Tracking (Carrier + Code Timing)

Tracking runs at the spread-symbol rate $\ell = 0, \dots, N_{\text{sym}} - 1$ and updates only on the 1024-chip data window of each symbol.

All divisions that use an ϵ term use $\epsilon = 10^{-18}$. Phase wrapping uses

$$\text{wrap}_{(-\pi, \pi]}(x) = (x + \pi) \bmod (2\pi) - \pi.$$

2.a Carrier Tracking (PLL/Costas with Frequency Bank)

The carrier tracker maintains two real-valued states: - θ_ℓ (rad): residual carrier phase at the start of the symbol's data window. - ω_ℓ (rad): residual carrier phase advance across the symbol's data window of duration $T_{\text{data}} = SF/R_c$.

Carrier compensation across chips uses the per-chip phasor

$$r_\ell[j] = \exp\left(-j\left(\theta_\ell + \frac{\omega_\ell}{SF}j\right)\right), \quad j = 0, \dots, SF - 1.$$

Per symbol, a frequency hypothesis bank searches small offsets around ω_ℓ : - Bank offsets: $\Delta f_h \in \{-4.0, -3.75, \dots, +4.0\}$ Hz with step 0.25 Hz. - Map to phase offsets: $\Delta\omega_h = 2\pi \Delta f_h T_{\text{data}}$. - Hypothesis: $\omega_{\ell,h} = \text{clamp}(\omega_\ell + \Delta\omega_h, [-\omega_{\text{max}}, +\omega_{\text{max}}])$, with $\omega_{\text{max}} = 2\pi \cdot 200 \text{ Hz} \cdot T_{\text{data}}$.

For each hypothesis h , rotate the combined (MRC) chip vector $\mathbf{u}_\ell[j]$ by $r_{\ell,h}[j]$ and compute: - **Preamble/Pilot** (W_0 known):

$$z_{\ell,h} = \sum_{j=0}^{SF-1} \mathbf{u}_\ell[j] \cdot r_{\ell,h}[j], \quad h_{\text{best}} = \arg \max_h |z_{\ell,h}|^2.$$

- **Data:**

$$R_{\ell,h}[m] = \sum_{j=0}^{SF-1} \mathbf{u}_{\ell}[j] \cdot r_{\ell,h}[j] \cdot W_m[j].$$

Use a 1024-point FHT to compute all $m \in [0, 1023]$ and retain $m \in [0, 255]$. Select

$$(h_{\text{best}}, \hat{m}_{\ell}) = \arg \max_{h,m \in [0,255]} |R_{\ell,h}[m]|^2.$$

Define confidence measures: - For data-symbol code confidence, let P_1 and P_2 be the largest and second-largest values of $|R_{\ell,h_{\text{best}}}[m]|^2$ over $m \in [0, 255]$, then

$$\text{conf}_{\text{code}} = \frac{P_1 - P_2}{|P_1| + \epsilon}.$$

- For frequency-bank confidence, let Q_1 and Q_2 be the largest and second-largest values of the selected hypothesis score (pilot: $|z_{\ell,h}|^2$; data: peak $|R_{\ell,h}[\cdot]|^2$), then

$$\text{conf}_{\text{freq}} = \frac{Q_1 - Q_2}{|Q_1| + \epsilon}.$$

Carrier state update uses a predict-then-correct structure: 1. **Predict:** the symbol spans the data window plus the guard/noise window. Let j_{ℓ} be the guard length in chips. The phase advance from the end of the data window to the start of the next data window is modeled by scaling with

$$s_{\ell} = 1 + \frac{j_{\ell}}{SF}.$$

With the selected $\omega_{\ell,h_{\text{best}}}$, predict:

$$\theta_{\ell} \leftarrow \text{wrap}_{(-\pi,\pi]}(\theta_{\ell} + \omega_{\ell,h_{\text{best}}} s_{\ell}).$$

2. **Correct:** - For preamble/pilot, compute $z_{\ell} = z_{\ell,h_{\text{best}}}$. - For data, compute $z_{\ell} = R_{\ell,h_{\text{best}}}[\hat{m}_{\ell}]$ and rotate it so the decided peak is real-positive: if $\Re\{z_{\ell}\} < 0$ then $z_{\ell} \leftarrow -z_{\ell}$. - Phase error:

$$e_{\ell} = \text{atan2}(\Im\{z_{\ell}\}, \Re\{z_{\ell}\} + \epsilon).$$

- Apply a PI correction with gains (K_p, K_i) and a mode-dependent scale:

$$\omega_{\ell+1} = \text{clamp}(\omega_{\ell} + (K_i \cdot s_{\text{dd}}) e_{\ell}, [-\omega_{\text{max}}, +\omega_{\text{max}}]),$$

$$\theta_{\ell} \leftarrow \text{wrap}_{(-\pi,\pi]}(\theta_{\ell} + (K_p \cdot s_{\text{dd}}) e_{\ell}),$$

where $s_{\text{dd}} = 1.0$ for preamble/pilots and $s_{\text{dd}} = 0.5$ for decision-directed data updates.

Pilot cycle-slip guard: - Let M_{pre} be a fixed preamble magnitude reference computed at initialization. - If a pilot symbol yields $\Re\{z_{\ell}\} < 0$ and $|z_{\ell}| > 0.25 M_{\text{pre}}$, then apply:

$$\theta_{\ell} \leftarrow \text{wrap}_{(-\pi,\pi]}(\theta_{\ell} + \pi), \quad A_i \leftarrow -A_i \quad \forall i,$$

and recompute the MRC weights w_i .

Frequency snap (anti-slip): - If $\text{conf}_{\text{freq}} \geq 0.15$ and the selected nonzero bank offset $\Delta f_{h_{\text{best}}}$ stays within ± 0.125 Hz of a constant value for 3 consecutive symbols, then: - if $|\Delta f_{h_{\text{best}}}| \geq 0.75$ Hz, update $\omega_{\ell} \leftarrow \text{clamp}(\omega_{\ell} + 2\pi \Delta f_{h_{\text{best}}} T_{\text{data}}, [-\omega_{\text{max}}, +\omega_{\text{max}}])$; - otherwise, do not change ω_{ℓ} ; - reset the snap state.

Loop gain design: - Let T_{upd} be the nominal per-symbol update period (data+guard). Define

$$T_{\text{upd}} = \frac{SF + j_{\text{avg}}}{R_c}, \quad j_{\text{avg}} = 40 + \frac{50}{2}.$$

- Gains are designed in discrete time via pole mapping. Given loop bandwidth B_L (Hz) and damping ζ , let $\omega_n = 2\pi B_L$ and

$$r = \exp(-\zeta \omega_n T_{\text{upd}}), \quad \phi = \omega_n \sqrt{1 - \zeta^2} T_{\text{upd}}.$$

The PI gains are

$$K_p = 1 - r^2, \quad K_i = 1 + r^2 - 2r \cos \phi.$$

Fixed values: - PLL: $B_L = 1.0$ Hz, $\zeta = 0.707$.

2.b Code Timing Tracking (Early/Late DLL)

The DLL tracks the chip sampling step in samples-per-symbol, step_{ℓ} .

- Nominal: $step_{\text{nom}} = SF \cdot OSF$.
- Clamp: $step_{\text{min}} = step_{\text{nom}}(1 - 2000 \cdot 10^{-6})$, $step_{\text{max}} = step_{\text{nom}}(1 + 2000 \cdot 10^{-6})$.
- Early/late spacing: 0.5 chips.

Let $\Delta_{\text{EL}} = 0.5 \cdot step_{\ell} / SF$ (samples) and sample Early/Prompt/Late with linear interpolation. After demasking, hop-delay compensation, carrier compensation (using the selected hypothesis h_{best}), and MRC combining, form the Early/Late prompt correlator outputs: - If the symbol uses W_0 (preamble/pilot): $z_E = \sum_j \mathbf{u}_E[j]$, $z_L = \sum_j \mathbf{u}_L[j]$. - If the symbol uses Walsh index $\hat{m}_{\ell}$ (data): $z_E = \sum_j \mathbf{u}_E[j] W_{\hat{m}_{\ell}}[j]$, $z_L = \sum_j \mathbf{u}_L[j] W_{\hat{m}_{\ell}}[j]$.

Define magnitudes $A_E = |z_E|$, $A_L = |z_L|$ and the normalized early/late discriminator:

$$d_{\ell} = \frac{A_E - A_L}{A_E + A_L + \epsilon}.$$

Map to a sample-domain timing error:

$$e_{\text{DLL},\ell} = d_{\ell} \cdot \frac{\Delta_{\text{EL}}}{2}.$$

The DLL applies a PI update:

$$step_{\ell+1} = \text{clamp} \left(step_{\ell} - (K_i^{\text{DLL}} \cdot s_{\text{dd}}^{\text{DLL}}) e_{\text{DLL},\ell}, [step_{\text{min}}, step_{\text{max}}] \right),$$

$$\Delta t_{\ell} = -(K_p^{\text{DLL}} \cdot s_{\text{dd}}^{\text{DLL}}) e_{\text{DLL},\ell}.$$

The fractional timing adjustment Δt_{ℓ} is applied to the symbol timeline. The decision-directed DLL scale is $s_{\text{dd}}^{\text{DLL}} = 0.25$ for data symbols and $s_{\text{dd}}^{\text{DLL}} = 1.0$ for preamble/pilots.

DLL gains use the same discrete-time pole-mapping design with fixed values: - DLL: $B_L = 0.6 \text{ Hz}$, $\zeta = 0.707$.

3. Despreading and Combining

For each spread symbol ℓ and each finger i , the receiver samples the 1024-chip data window, de-masks it, applies hop-delay phase compensation, and then combines fingers with MRC.

1. **Chip Sampling with Linear Interpolation:** Let $t_{i,\ell}$ be the tracked symbol start time (samples) for finger i and symbol ℓ , and let the tracked samples-per-chip be $\Delta_{\text{chip}} = step_{\ell} / SF$. For each chip index $j = 0, \dots, SF - 1$, sample from the matched-filtered signal $y[n]$ at

$$p = t_{i,\ell} + j \Delta_{\text{chip}}.$$

With $n_0 = \lfloor p \rfloor$ and $a = p - n_0 \in [0, 1)$, the interpolated sample is

$$\mathbf{y}_{i,\ell}[j] = (1 - a) y[n_0] + a y[n_0 + 1].$$

2. **De-masking:** multiply by the locally generated chip mask:

$$\mathbf{u}_{i,\ell}[j] = \mathbf{y}_{i,\ell}[j] \cdot C_{\text{seq}}[\ell \cdot SF + j].$$

For the second preamble symbol ($\ell = 1$), apply the Barker-2 sign:

$$\mathbf{u}_{i,1}[j] \leftarrow -\mathbf{u}_{i,1}[j].$$

3. **Hop-delay phase compensation:** for the i -th finger, let Δd_i be the integer delay (samples) relative to the main finger (Step C.1). For symbol ℓ , compute

$$c_{i,\ell} = \exp \left(+j 2\pi f_{\ell} \frac{\Delta d_i}{F_s} \right),$$

and apply

$$\mathbf{u}_{i,\ell}[j] \leftarrow \mathbf{u}_{i,\ell}[j] \cdot c_{i,\ell}.$$

The receiver applies this compensation to all branches that enter MRC, including prompt and the DLL Early/Late branches.

All fingers are combined using Maximal-Ratio Combining (MRC) by a weighted sum in the chip domain to obtain the de-masked chip vector for that symbol:

$$\mathbf{u}_\ell[j] = \sum_i w_i \cdot \mathbf{u}_{i,\ell}[j], \quad w_i = \frac{A_i^*}{\sum_k |A_k|^2}$$

where A_i is the estimated complex channel gain for the i -th finger. A_i is initialized from the two preamble symbols and is updated on every pilot symbol with an exponential smoother with $\alpha = \frac{1}{16}$:

$$\tilde{A}_i^{(r)} = \frac{1}{SF} \sum_{j=0}^{SF-1} \mathbf{u}_{i,\ell_{\text{pilot}}(r)}[j] \cdot r_{\ell_{\text{pilot}}(r),h_{\text{best}}}[j], \quad A_i^{(r)} = (1 - \alpha)A_i^{(r-1)} + \alpha\tilde{A}_i^{(r)}, \quad \alpha = \frac{1}{16}.$$

Step D: Walsh Demodulation and Polar (CA-SCL) Decoding

1. Pilot-aided Carrier Tracking

Pilot symbols (known W_0) provide a coherent reference for the carrier tracker and channel smoother defined in Step C.2 and Step C.3.

2. Walsh Matched Filtering (FHT)

For each data symbol $\ell = \ell_{\text{data}}(q)$ ($q = 0, \dots, N_{\text{data}} - 1$) and each symbol-rate frequency hypothesis k (selected from the bank in Step C), compute the M_W -way correlation on the carrier-compensated chip vector:

$$R_{\ell,k}[m] = \sum_{j=0}^{SF-1} \mathbf{u}_{\ell,k}[j] \cdot W_m[j], \quad m \in [0, 255]$$

Use a 1024-point Fast Walsh-Hadamard Transform (FHT) to compute the full correlation for $m \in [0, 1023]$, and then truncate to the first M_W results for $m \in [0, 255]$. Select the hypothesis k_{best} for this symbol using the peak energy criterion (Step C), and pass $R_{\ell,k_{\text{best}}}[m]$ to the soft-demapper.

3. Soft Information (LLR) Generation

Apply a decision-directed common phase rotation before soft-demapping to make the decided peak real-positive:

$$\tilde{R}_\ell[m] = R_{\ell,k_{\text{best}}}[m] \cdot e^{-j \arg(R_{\ell,k_{\text{best}}}[\hat{m}_\ell])}.$$

Let the metric be $D_\ell[m] = \Re\{\tilde{R}_\ell[m]\}$. If the decision-directed rotation is undefined (zero magnitude), use $D_\ell[m] = \Re\{R_{\ell,k_{\text{best}}}[m]\}$. For the t -th bit within each symbol ($t = 0, \dots, k - 1$), calculate the max-log LLR:

$$LLR_{qk+t} = \max_{m: ((m \gg (k-1-t)) \& 1) = 0} D_\ell[m] - \max_{m: ((m \gg (k-1-t)) \& 1) = 1} D_\ell[m]$$

The $N_{\text{data}} \cdot k = M$ LLRs correspond to the interleaved coded bits $B^{(\pi)}$. Apply the inverse permutation to deinterleave them before decoding:

$$LLR_i = LLR_{\pi^{-1}(i)}^{(\pi)}.$$

4. CA-SCL Polar Decoding and Output

Perform CRC-aided SCL (CA-SCL) Polar decoding on the deinterleaved LLRs to obtain the information bits $\hat{U}$. Use the existing CRC-32C (embedded inside U) as the list selection rule: choose the best-metric candidate that passes CRC; if none pass CRC, choose the best-metric candidate. Then perform a final CRC-32C check; if it passes, output $\hat{U}$ and truncate the payload according to the **Len** field in the Header. If it fails, discard the packet.